
boolean



useEffect

Un nuovo hook!

React Hooks

Cosa sono i React hooks? (Ripassino)

I React hooks sono delle **funzioni speciali** forniteci da React, che ci permettono di gestire moltissimi aspetti delle nostre applicazioni.

Ce ne sono molti ed ognuno ha il suo scopo. Possiamo anche creare degli **hook custom** ossia delle funzioni personalizzate da poter riutilizzare nei nostri progetti.

Tra quelli principali forniti da React, spiccano certamente **useState** e **useEffect.**

Per utilizzarli, basta importarli dalla libreria React in cima ai componenti

```
1 import {useState, useEffect} from 'react';
```

React Hooks

useState hook (ormai lo conosciamo bene)

L'hook **useState** ci permette di dichiarare una variabile "dinamica" (detta **variabile di stato**) che React terrà particolarmente d'occhio, cosicchè se il suo valore dovesse cambiare, il componente (ed eventualmente la sua UI)cambierà e si aggiornerà di conseguenza!

useState ha le seguenti regole:

- si **invoca** passandole il **valore iniziale** della variabile
- si **destruttura** con un array i cui due elementi sono rispettivamente:
    - il **nome della variabile** (es: `count`)
    - la **funzione** da utilizzare **per modificarne il valore** (es: `setCount`)

```
1 import {useState} from 'react';
2
3 function App(){
4   const [count, setCount] = useState(0);
5 }
6
7 export default App
```

React Hooks

useEffect hook (intro)

L'hook **useEffect** aggiunge ancora più potenza ai nostri codici, perchè permette di intervenire in momenti specifici e/o di reagire in maniera mirata a ciò che succede alle nostre variabili di stato!

Per prima cosa, vediamo come si scrive, poi capiremo come e quando utilizzarlo!

L'hook **useEffect** va importato da react come tutti gli hooks.

Possiamo poi invocarlo e passare **2 argomenti:**

-Una **callbackFunction** in cui eseguire del codice a piacimento

- Un **Array di dipendenze** che lo aiuterà a capire **quando** la funzione dovrà essere eseguita!

```
1 import { useEffect } from 'react';
2
3 function App(){
4   useEffect(() => {}, [])
5 }
6
7 export default App
```

React Hooks

useEffect hook (dipendenze)

La funzione all'interno di useEffect **viene sempre eseguita almeno una volta**, all'avvio del componente.

Ma come facciamo a dirgli quali altre volte essere eseguita? Grazie **all'array di dipendenze!**

All'interno di questo array, elenchiamo tutte le variabili **da cui dipende** la logica al suo interno.

Ogni volta che il valore di una delle dipendenze cambia, useEffect viene eseguito di nuovo

```jsx
import { useState, useEffect } from "react";

export default function App() {
  const [counter, setCounter] = useState(0);

  function handleIncrement() {
    setCounter((current) => current + 1);
  }

  useEffect(() => {
    console.log("Eseguita!");
  }, [counter]);

  return (
    <div>
      <div>{counter}</div>
      <button onClick={handleIncrement}>
        Incrementa
      </button>
    </div>
  );
}
```



LIVE CODING

useEffect



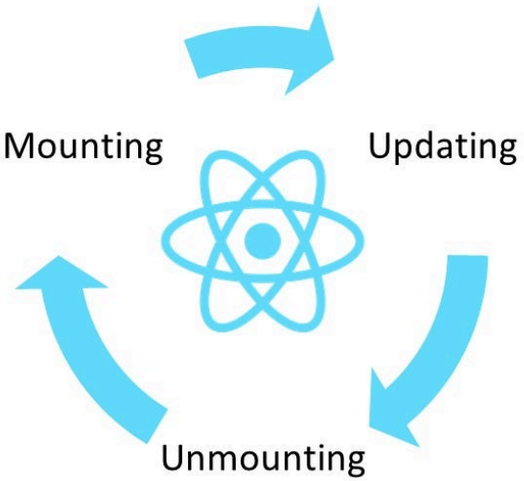
Component LifeCycle

useEffect e il ciclo di vita dei componenti

Component LifeCycle

useEffect hook (ciclo di vita)

L'hook useEffect è anche utilizzato per gestire **il ciclo di vita** del componente.



Ma cos'è il **ciclo di vita** di un componente?

Ogni componente attraversa diverse fasi:

- Viene **montato**, quando lo visualizziamo la prima volta in pagina
- Viene **aggiornato**, quando i suoi dati cambiano
- Viene **smontato** , quando non serve più sul browser. *(ad es, cambiamo pagina)*

Component LifeCycle

useEffect hook (mounting)

Tutti gli **useEffect** all'interno di un componente **vengono immediatamente eseguiti** quando il componente viene montato.

Questo significa che i nostri useEffect vengono sempre eseguiti **almeno una volta!**

Quando vogliamo eseguire qualcosa **solo all'avvio del componente** (ad esempio all'avvio della pagina) possiamo **lasciare vuoto l'array delle dipendenze!**



```
1  import { useState, useEffect } from "react";
2
3  export default function App() {
4    //...
5
6    useEffect(() => {
7      console.log("Il componente è stato montato!");
8    }, []);
9
10   return (
11     <div>
12       //...
13     </div>
14   );
15 }
```

Component LifeCycle

useEffect hook (updating)

Tutti i componenti, si aggiornano quando i loro stati cambiano.

Quando un componente si aggiorna, ogni useEffect si domanda se deve eseguire nuovamente il suo codice.

Per rispondersi, guarda l'array delle dipendenze!

Se una dipendenza si è aggiornata, allora useEffect esegue nuovamente la sua funzione

```
1  // Immaginiamo 2 stati
2  const [counter, setCounter] = useState(0);
3  const [text, setText] = useState('Lorem ipsum')
4
5  // Solo all'avvio
6    useEffect(() => {
7      console.log("Solo all'avvio")
8    }, []);
9
10 // All'avvio e ogni volta che cambia `counter`
11 // ma non se cambia text
12   useEffect(() => {
13     console.log(`Il contatore è: ${counter}`);
14   }, [counter]);
15
16 // All'avvio e se cambia `text`
17 // ma non se cambia cambia `counter`
18   useEffect(() => {
19     console.log(`Qualcuno ha cambiato il testo: $
20   }, [text]);
21
22 // All'avvio e se cambia `text` o `counter`
23   useEffect(() => {
24     console.log("Qualcosa è cambiato!");
25   }, [text, counter]);
```

Component LifeCycle

useEffect hook (unmounting)

useEffect può anche eseguire del codice quando un componente viene **smontato.**

Per farlo, bisogna fare in modo che la funzione di useEffect, **restituisca un'altra funzione** (detta **cleanup function**)

Il codice di questa funzione verrà eseguito una volta che il componente sarà rimosso dalla pagina!

```
1  useEffect(() => {
2     // Questo codice viene eseguito
3     // in base alle dipendenze, come sempre
4     console.log('Componente montato');
5
6     // Questa funzione viene eseguita
7     // quando il componente viene smontato,
8     // a prescindere dalle dipendenze
9     return () => {
10      console.log('Componente smontato');
11    };
12  }, []);
```



L'array delle dipendenze non influisce sulla cleanup function (può essere vuoto o pieno)

React Hooks

useEffect hook (casi d'uso)

L'hook useEffect viene spesso utilizzato per gestire i "**side effect**", ovvero tutte le interazioni tra un componente React e il mondo esterno al componente.

I "**side effect**" possono includere operazioni come:

- il **fetching di dati** da un server
- la gestione di **timer**
- l'**accesso manuale ad elementi del DOM**

Tutte queste operazioni verranno eseguite da useEffect in diversi momenti, **in base a come usiamo l'array di dipendenze!**

ESERCIZIO